

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

## Objectifs pédagogiques

À l'issue de ce TP, l'étudiant est capable de :

- Concevoir un mini-projet Python complet de A à Z
- Structurer un projet en plusieurs fichiers
- Implémenter une interface utilisateur interactive (menu)
- Appliquer les bonnes pratiques : fonctions, docstrings, gestion d'erreurs
- Produire un projet pédagogique réutilisable en classe
- Importer et exporter des données au format CSV

## Plan du TP

| Phase                  | Durée  | Activité                    |
|------------------------|--------|-----------------------------|
| Présentation du projet | 10 min | Architecture et structure   |
| Exercice 1             | 15 min | Créer la base questions CSV |
| Exercice 2             | 15 min | Module de chargement CSV    |
| Exercice 3             | 20 min | Interface menu interactive  |
| Exercice 4             | 15 min | Tirage aléatoire et scoring |
| Exercice 5             | 15 min | Export des résultats        |

## Présentation du mini-projet

Sujet : Créer un Quiz pédagogique interactif sur les notions d'informatique vues dans le module.

Fonctionnalités attendues :

1. Charger des questions depuis un fichier CSV (4 catégories, 15+ questions)
2. Poser les questions dans un ordre aléatoire
3. Gérer plusieurs catégories (Architecture, Algorithmique, Python, Réseaux)
4. Chronométrer le quiz
5. Calculer le score et afficher les résultats
6. Exporter les résultats dans un fichier CSV
7. Interface en ligne de commande avec menu

```

flowchart TD
    subgraph Projet[Structure du Projet Quiz]
        A[quiz.py] --> B[questions.csv]
        A --> C[resultats.csv]
    end
    A --> D[Menu interactif]
    D --> E[Lancer le quiz]
    D --> F[Afficher les règles]
    D --> G[Quitter]
    E --> H[Charger les questions]
    H --> I[Filtrer par catégorie]
    I --> J[Mélanger aléatoirement]
    J --> K[Poser les questions]
    K --> L[Calculer le score]
    L --> M[Afficher les résultats]
    M --> N[Exporter en CSV]

```

## Barème complet

| Partie                      | Points        | Détail                                           |
|-----------------------------|---------------|--------------------------------------------------|
| Base de questions CSV       | 4 pts         | 4 catégories, 15+ questions, format correct      |
| Module de chargement        | 4 pts         | Fonctions charger, filtrer, mélanger             |
| Interface menu              | 4 pts         | Menu clair, navigation fluide, gestion d'erreurs |
| Tirage aléatoire et scoring | 4 pts         | Ordre aléatoire, score, mention, chronomètre     |
| Export des résultats        | 4 pts         | Export CSV propre, horodatage                    |
| <b>Total</b>                | <b>20 pts</b> |                                                  |

## Exercice 1 — Créer la base questions CSV (15 min)

Énoncé : Créer un fichier questions.csv avec au moins 15 questions réparties en 4 catégories (Architecture, Algorithmique, Python, Réseaux). Format : Catégorie, Question, Réponse.

Corrigé :

```

import csv

questions = [
    ["Architecture", "Que signifie CPU ?", "Central Processing Unit"],
    ["Architecture", "La RAM est-elle volatile ?", "Oui"],
    ["Architecture", "Quel modèle est à la base des ordinateurs modernes ?", "von Neumann"],
    ["Algorithmique", "Quelle structure suit le principe LIFO ?", "Pile"],
    ["Algorithmique", "Quelle structure suit le principe FIFO ?", "File"],
    ["Algorithmique", "Quelle est la complexité du tri à bulles ?", "O(n²)"],
    ["Python", "Quelle fonction affiche du texte en Python ?", "print"],
    ["Python", "Quel mot-clé définit une fonction ?", "def"],
    ["Python", "Quel type retourne input() ?", "str"],
    ["Python", "Comment gérer les erreurs en Python ?", "try/except"],
    ["Réseaux", "Que signifie IP ?", "Internet Protocol"],
    ["Réseaux", "Quel protocole sécurise HTTP ?", "HTTPS"],
    ["Réseaux", "Que signifie DNS ?", "Domain Name System"],
    ["Réseaux", "Quel port utilise HTTP ?", "80"],
    ["Réseaux", "Que signifie LAN ?", "Local Area Network"],
]

with open("questions.csv", "w", newline="", encoding="utf-8") as f:

```

```
writer = csv.writer(f)
writer.writerow(["Categorie", "Question", "Reponse"])
writer.writerows(questions)

print(f"{len(questions)} questions créées dans questions.csv")
```

## Exercice 2 — Module de chargement CSV (15 min)

Énoncé : Créer les fonctions de chargement des questions dans `quiz.py` :

1. `charger_questions(fichier)` — charge le CSV et retourne une liste de dictionnaires
2. `filtrer_par_categorie(questions, categorie)` — filtre les questions par catégorie
3. `melanger_questions(questions)` — mélange aléatoirement

Corrigé :

```
import csv
import random
import time
from datetime import date

def charger_questions(fichier):
    """Charge les questions depuis un fichier CSV."""
    questions = []
    try:
        with open(fichier, "r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            for ligne in reader:
                questions.append({
                    "categorie": ligne["Categorie"],
                    "question": ligne["Question"],
                    "reponse": ligne["Reponse"]
                })
    except FileNotFoundError:
        print(f"Erreur : fichier {fichier} introuvable.")
        return []
    return questions

def filtrer_par_categorie(questions, categorie):
    """Filtre les questions par catégorie."""
    if categorie == "Toutes":
        return questions
    return [q for q in questions if q["categorie"] == categorie]

def melanger_questions(questions):
    """Mélange les questions aléatoirement."""
    random.shuffle(questions)
    return questions
```

## Exercice 3 — Interface menu interactive (20 min)

Énoncé : Créer le menu principal du quiz avec les options suivantes :

1. Lancer le quiz (choix de catégorie)
2. Afficher les règles
3. Quitter

Le menu doit être clair, robuste et gérer les erreurs de saisie.

```

flowchart TD
    A[Écran d'accueil] --> B[Menu principal]
    B --> C[1. Lancer le quiz]
    B --> D[2. Règles]
    B --> E[3. Quitter]
    C --> F[Choisir catégorie]
    F --> G[Saisir nom]
    G --> H[Lancer le quiz]
    H --> I[Afficher résultats]
    I --> J[Exporter ?]
    J -->|Oui| K[Exporter CSV]
    J -->|Non| B
    K --> B
    D --> L[Afficher règles]
    L --> B
    E --> M[Fin]

```

Corrigé :

```

def menu_principal():
    """Affiche le menu principal et gère les choix."""
    questions = charger_questions("questions.csv")
    if not questions:
        return
    categories = ["Toutes", "Architecture", "Algorithmique", "Python", "Réseaux"]
    while True:
        print("\n" + "="*40)
        print("1. Lancer le quiz")
        print("2. Règles")
        print("3. Quitter")
        choix = input("Votre choix : ")
        if choix == "1":
            for i, cat in enumerate(categories, 1):
                print(f"{i}. {cat}")
            try:
                c = int(input("Catégorie (1-5) : "))
                cat = categories[c - 1] if 1 <= c <= 5 else "Toutes"
            except (ValueError, IndexError):
                cat = "Toutes"
            qf = filtrer_par_categorie(questions, cat)
            print(f"Questions : {len(qf)} dans '{cat}'")
            lancer_quiz(qf, input("Votre nom : "))
        elif choix == "2":
            print("• Réponse textuelle • 4 catégories • Score /20 • Export CSV")
        elif choix == "3":
            print("Au revoir !")
            break
        else:
            print("Choix invalide.")

```

## Exercice 4 — Tirage aléatoire et scoring (15 min)

Énoncé : Ajouter les fonctions de quiz avec tirage aléatoire, scoring et mention. Le programme mélange les questions, pose chaque question, vérifie la réponse, et attribue une mention.

Corrigé :

```

def poser_question(question):
    """Pose une question et retourne True si bonne réponse."""
    print(f"\n[{question['categorie']}] {question['question']}")
    reponse = input("Votre réponse : ").strip().lower()
    reponse_attendue = question['reponse'].strip().lower()

```

```

if reponse == reponse_attendue:
    print("✓ Correct !")
    return True
else:
    print(f"x Faux. Réponse : {question['reponse']}")
    return False

def lancer_quiz(questions, nom_eleve=""):
    """Lance le quiz avec mélange aléatoire et chronomètre."""
    if not questions:
        print("Aucune question à poser.")
        return

    questions = melanger_questions(questions)
    score = 0
    total = len(questions)
    debut = time.time()

    print(f"\n=== Quiz lancé ! ({total} questions) ===")
    for i, q in enumerate(questions, 1):
        print(f"\nQuestion {i}/{total}")
        if poser_question(q):
            score += 1

    temps = time.time() - debut
    afficher_resultats(score, total, temps)
    exporter = input("\nExporter les résultats ? (o/n) : ")
    if exporter == "o" and nom_eleve:
        exporter_resultats(score, total, temps, nom_eleve)

def afficher_resultats(score, total, temps):
    """Affiche les résultats avec mention."""
    pourcentage = (score / total * 100) if total > 0 else 0
    print("\n" + "="*40)
    print(" RÉSULTATS")
    print("="*40)
    note = round(score / total * 20, 2) if total > 0 else 0
    print(f"Score : {score}/{total} ({pourcentage:.1f}%)")
    print(f>Note : {note}/20")
    print(f"Temps : {temps:.1f}s")

    if pourcentage == 100:
        mention = "Parfait !"
    elif pourcentage >= 80:
        mention = "Très bien !"
    elif pourcentage >= 60:
        mention = "Bien !"
    elif pourcentage >= 40:
        mention = "Peut mieux faire."
    else:
        mention = "À revoir."
    print(f"Mention : {mention}")

```

## Exercice 5 — Export des résultats (15 min)

Énoncé : Ajouter la fonction d'export qui sauvegarde les résultats (date, nom, score, total, pourcentage, temps) dans `resultats.csv` avec horodatage automatique.

```

flowchart LR
    A[Quiz terminé] --> B{Exporter ?}
    B -->|Oui| C[Ouvrir resultats.csv]
    B -->|Non| D[Retour menu]

```

```

C --> E{Fichier vide ?}
E -->|Oui| F[Écrire en-tête]
E -->|Non| G[Ignorer]
F --> H[Ajouter résultat]
G --> H
H --> I[✓ Exporté !]
I --> D

```

Corrigé :

```

def exporter_resultats(score, total, temps, nom_eleve, nom_fichier="resultats.csv"):
    """Exporte les résultats du quiz dans un fichier CSV."""
    with open(nom_fichier, "a", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        if f.tell() == 0:
            writer.writerow(["Date", "Eleve", "Score", "Total",
                             "Pourcentage", "Note_20", "Temps_s"])
        note = round(score / total * 20, 2) if total > 0 else 0
        pourcentage = f"{score/total*100:.1f}" if total > 0 else "0"
        writer.writerow([date.today(), nom_eleve, score, total,
                         pourcentage, note, f"{temps:.1f}"])
    print(f"Résultats exportés dans {nom_fichier}")

def main():
    """Point d'entrée du programme."""
    afficher_accueil()
    try:
        with open("questions.csv", "r", encoding="utf-8"):
            pass
    except FileNotFoundError:
        print("\n⚠ Fichier questions.csv introuvable. Lancez d'abord l'exercice 1.")
        return
    menu_principal()

if __name__ == "__main__":
    main()

```

## QCM d'auto-évaluation

1. Quelle fonction permet de mélanger aléatoirement une liste ? a) `random.shuffle()` ✓ b) `random.choice()` c) `random.sample()`
2. Quel est le bon format pour un fichier CSV ? a) Données séparées par des virgules ✓ b) Données séparées par des espaces c) Données en format JSON
3. Que fait `csv.DictReader()` ? a) Lit chaque ligne comme un dictionnaire ✓ b) Écrit des dictionnaires c) Convertit une liste en dictionnaire
4. Dans le barème du mini-projet, combien de points vaut la base de questions ? a) 2 points b) 4 points ✓ c) 6 points
5. Quel module Python permet d'obtenir la date du jour ? a) `datetime` ✓ b) `time` c) `calendar`

## Grille d'évaluation du mini-projet

| Critère                 | Poids | Non acquis (0)                | Partiel (0.5)            | Acquis (1)     |
|-------------------------|-------|-------------------------------|--------------------------|----------------|
| Fonctionnalité complète | 20 %  | Le programme ne s'exécute pas | Fonctionne partiellement | Fonctionne par |

|                       |      |                  |                               |                  |
|-----------------------|------|------------------|-------------------------------|------------------|
| Code structuré        | 15 % | Pas de fonctions | Quelques fonctions            | Fonctions bien   |
| Gestion des erreurs   | 15 % | Aucune gestion   | Gestion partielle             | try/except + va  |
| Fichiers CSV          | 15 % | Pas de CSV       | Lecture ou écriture seulement | Lecture et écrit |
| Interface utilisateur | 15 % | Pas de menu      | Menu basique                  | Menu clair et fo |
| Documentation         | 10 % | Aucune           | Commentaires minimales        | Docstrings + c   |
| Originalité du projet | 10 % | Copie conforme   | Légères adaptations           | Projet personna  |

## Déroulement séance — 1h30 (90 min)

| Phase                               | Durée         | Total cumulé |
|-------------------------------------|---------------|--------------|
| Présentation du projet              | 10 min        | 10 min       |
| Exercice 1 — Base de questions      | 15 min        | 25 min       |
| Exercice 2 — Module chargement      | 15 min        | 40 min       |
| Exercice 3 — Interface menu         | 20 min        | 60 min       |
| Exercice 4 — Tirage et scoring      | 15 min        | 75 min       |
| Exercice 5 — Export et finalisation | 15 min        | 90 min       |
| <b>Total</b>                        | <b>90 min</b> |              |

## Note pédagogique ENS

Ce mini-projet est l'aboutissement du module. Il mobilise toutes les compétences acquises : variables, conditions, boucles, fonctions, listes, dictionnaires, fichiers CSV, et gestion d'erreurs.

- Pour les futurs enseignants : Ce quiz est un outil directement réutilisable. Ils peuvent adapter les questions à leur matière (histoire, langues, sciences) et utiliser l'export CSV pour suivre la progression des élèves.
- En classe de secondaire : Simplifier en ne gardant qu'une seule catégorie. L'important est que l'élève comprenne le flux : fichier → programme → résultat → export.
- Prolongements : Interface graphique (tkinter), classement entre élèves, niveaux de difficulté, mode multijoueur.